

agent-verdict

A short code walkthrough

Branch m2

Commit b542e865bd

2026-09-22

agent-verdict: a short code walkthrough

This is a condensed version of docs/walkthrough/code-walkthrough.md, cut from about 47 pages to about 10. It covers the same facts, verified against the code, but skips most excerpts and secondary detail. Read the long version for anything you need in full.

1. What Verdict is

Claude Code, Anthropic's command line coding assistant, ends a turn with a message like "Fixed the bug, all tests pass." Nobody checks that claim. A command can fail three steps earlier, the model can move on, and the final message can still say the work is done.

Verdict's answer is to write down what actually happened, as it happens, into a private file on your own computer, and later compare that record against the claim. Claude Code lets a plugin register hooks: small programs it runs at fixed moments in a session. Verdict registers on six of those moments and appends one line per event to a local file. At the end of a turn, the plan is to hand the record and the final message to a small model, called Jev (a verifier model from the provider TypeSafe), and ask whether the claim matches the record. Jev only ever reads the assistant's final message and the extracted evidence. It never runs commands and never sees your files.

Today (M1, tag m1): only the recorder exists. Verdict records every event, tags some of them (is this a check, does this look like a failure that exited zero), and extracts claim sentences from the final message. It makes no judgment and sends nothing over the network. The README states this plainly: "v0.1 collector. Records locally, sends nothing."

Coming (M2, branch m2, in progress): the verifier. At Stop, Verdict will assemble the evidence into a verification span (the current prompt plus recent ones), decide whether asking Jev is worth it, ask a small number of yes/no questions, and apply a policy to the answers. The span-building code (span.py) is the first piece of M2 and already exists; the model call itself does not.

2. How an event flows

```
Claude Code event
|
v (Claude Code runs the registered command for this event)
run.sh
|
v (picks a Python interpreter, honors the kill switch, execs the entry point)
verdict_hook.py
|
v (reads JSON from stdin, turns it into one typed event object)
parsers.py
|
v (builds one ledger row: gates, redact, truncate, tag)
recorders.py
|
v (one os.write of one JSON line, under a file lock)
ledger.py
|
v
~/verdict/events/<session_id>.jsonl
```

JSON (JavaScript Object Notation, a plain-text format for structured data) is what Claude Code sends in and what the ledger stores, one object per line.

Each arrow also has a side effect not drawn above: every invocation, whatever else happens, writes one line to `~/verdict/hook.log` through `logsafe.py`, recording only the event name, the outcome, and the elapsed time in milliseconds, never tool content.

3. The files, one paragraph each

Decision ids like D-003 refer to entries in docs/DECISIONS.md, the record of why a non-obvious choice was made.

The launcher and entry point

`plugin/hooks/run.sh`. A 50-line POSIX (a common Unix shell standard) script. It checks the `VERDICT_DISABLE` kill switch first, unsets `PYTHONPATH/PYTHONHOME/PYTHONSTARTUP` so nothing hijacks how Python starts, picks an interpreter (an env var, then a saved path, then `python3` on `PATH`), and execs it with `-S`, never `-E` (D-019: `-E` measured about 3x slower `stdlib` import on Apple's Python). The one design choice that matters is that it never prints anything, on any path, because Claude Code can read a hook's `stdout` as a JSON decision. If no usable interpreter exists, it exits 0 silently rather than showing the user an error.

`plugin/hooks/verdict_hook.py`. The entry point. It reads `stdin` (capped at 5MB), parses the payload, and always exits 0 with empty `stdout`, even on a crash (D-004: hooks are synchronous and side-effect-free on failure). It checks the kill switch and the plugin's own mode: `off` setting before importing anything, and only imports the parser and recorder lazily, inside the functions that need them, per D-028's measured import cost. If anything raises, it logs the exception's class name only, never its text, and still exits 0.

`plugin/hooks/hooks.json`. The manifest that tells Claude Code which six events call `run.sh`, with what argument and timeout: `SessionStart`, `UserPromptSubmit`, `PostToolUse` (matched to `Bash|Write|Edit|NotebookEdit|WebFetch|Agent|mcp_.*`), `PostToolUseFailure` (matched to everything), `Stop`, `SessionEnd`. The one choice that matters is the `PostToolUse` matcher: read-only tools like `Read` and `Grep` are excluded because a read success is weak evidence and every recorded event costs a process launch. If the JSON is malformed, the plugin simply never runs.

`plugin/.claude-plugin/plugin.json` and `.claude-plugin/marketplace.json`. The plugin's identity card (name, version, the three-mode `shadow/enforce/off` setting, D-014) and the marketplace file that makes the repository installable with `/plugin marketplace add` and `/plugin install`. If either is malformed, Claude Code's own `claude plugin validate` catches it before install; there is no runtime failure mode.

`plugin/hooks/verdict_hot/`, the hot path package

`__init__.py`. Twelve lines: marks the package and holds `SCHEMA_V` and `PLUGIN_VERSION`, written onto every row. The choice that matters is that every module inside this package imports its neighbors with relative imports, which is what lets the identical file tree also work as `agent_verdict.verdict_hot` inside the CLI (command line interface) package. A broken import here means the hook never runs at all, silently, since `run.sh` swallows all output.

`paths.py`. Answers "where does Verdict's data live" and enforces that it is private: `~/ .verdict` (or `$VERDICT_HOME`), mode 0700, with an explicit `chmod` after `mkdir` because `umask` can otherwise weaken it. It validates session ids against a short deny-list (`/`, `\`, `.`, `..`, control characters, length over 128) before they become file names, which is what stops a crafted session id from writing outside the data directory. Everything created beneath the root refuses to be a symlink; the root itself may be one, so a user can point their data at another disk. A `ValueError` here propagates up and the row is lost, silently, with only a log line.

`ledger.py`. Appends one JSON line per row, atomically, using one `os.write` under an exclusive `flock`, `O_APPEND`, and `O_NOFOLLOW` so the write can never go through a symlink (D-005: JSON Lines, not SQLite, because parallel hook processes would otherwise contend for a database lock). The detail that matters is that the lock is what actually prevents two processes from interleaving halves of one row, since a write can be split by the kernel past the atomic pipe size. If the directory is unwritable or the disk is full, the `OSError` propagates to `verdict_hook.py`, which logs exception and still exits 0.

`logsafe.py`. Writes the one-line operations log, `hook.log`, and guarantees the line never carries tool content or a key. `scrub()` removes configured provider key values, authorization headers, and `sk--`-shaped tokens, and it scrubs the row's Python structure before serializing it to JSON, not after, so a replacement can never land across a quote and corrupt the line. It rotates the file at 10MB under its own lock. `log_invocation` never raises; if scrubbing itself fails, nothing is logged for that call rather than risking a leak.

`textnorm.py`. Cleans text before storage: Unicode NFKC (a standard way of folding look-alike characters into one form) normalization plus stripping of zero-width, bidirectional-control, and ANSI escape characters, which is what stops hidden or reordered text ("Trojan Source" attacks) from landing in the ledger. Its other job, `truncate_anchored`, keeps the head and the tail of long text and drops the middle, because the most useful line of a failure is usually near the end. Neither function can raise; any string is valid input.

`redact.py`, `_redact_rules.py`, `_redact_filters.py`. The redactor (D-017: one `redact()` function, generated from a pinned gitleaks rule set plus local entropy rules, called at exactly the point text is about to be written).

`_redact_rules.py` is generated by `scripts/gen_redact.py` and must never be hand-edited; it holds 232 pattern rules, each gated by a keyword prefilter so most calls only run the four rules with no keyword at all. `redact.py` compiles patterns lazily and caches them, scans text in overlapping 64KB windows to bound worst-case regex time, and merges overlapping matches from different rules so a secret can never be left half-redacted. `_redact_filters.py` holds the checks that stop over-eager rules from redacting a git hash, a UUID (universally unique identifier), an SSH (secure shell) public key, or an ordinary English word. If anything in the pipeline raises, `redact()` returns the literal string `[redaction failed]` and a `-1` hit count; it never returns the original text, and the row is flagged `redaction_failed: true`.

`policy.py` and `plugin/policies/default.json`. Load the tuning knobs: how much text to keep, which paths and commands count as touching a credential, which commands count as a check, which words count as a success claim, and (new in M2) the verification span length and the acknowledgment threshold. JSON, not YAML (a more permissive text format for structured data), because YAML needs a third-party parser (D-013). Value types are `typing.NamedTuple`, never `dataclasses` (D-028: `dataclasses` cost about 8ms per process on Python 3.9 by pulling in `inspect`, `ast`, `dis`, and `tokenize`; a `NamedTuple` costs about 3ms). A malformed policy file raises `PolicyError` here and does not fail open; the fallback to the packaged default happens one layer up, in `recorders.py`, so a broken user override never stops a row from being written.

`gates.py`. Three pure, policy-driven questions: does this tool call touch a credential file or print one (`is_never_send`), is this command a test/build/lint/type check (`is_check`), and does this output look like a failure that nonetheless exited zero (`is_soft_fail_candidate`). Every pattern list is joined into one compiled regular expression, cached, which cut compiled patterns on a typical event from about 60 down to 2. `is_never_send` checks several possible path-bearing keys, not just `file_path`, because an MCP (Model Context Protocol, the standard for external tool servers) tool's schema is unknown and a narrower check once let a credential path through. A malformed user pattern is dropped individually rather than crashing the whole check.

`claims.py`. Pulls success-claiming sentences out of the assistant's final message, in code, not by asking a model, because the claim list is an input to the later model question, not its output. It strips code fences and quotes first (so a claim shown inside a code block is not mistaken for one the assistant is making), splits into sentences, and keeps only sentences with an unnegated success verb ("fixed", "passing", and ten more), capped at six. Negation ("did not pass") is checked over a six-word lookback against 21 cues. If the message is empty or has no code fences to strip, the function just returns no claims; it does not raise.

`parsers.py`. Turns a raw JSON payload into one of six typed event objects, or raises `ParseError` naming the offending field. It was written against real captured payloads, not documentation, which is why three unusual facts are hard-coded: `prompt_id` is absent on `SessionStart`, `agent_id/agent_type` were never observed at the top level, and `cc_effort` is always written `null`. Unknown extra fields are ignored, so a future Claude Code release cannot break the parser by adding one. `parse_exit_code` implements D-002: an exit code is a fact, extracted only when the error's first line is exactly `Exit code N`, never guessed and never a model's opinion. A payload that fails to parse is skipped entirely; no row is written for it, and the reason is logged as skipped.

`_tool_output.py`. Converts a tool's response into text safe to store, before the redactor ever sees it, because `redact()` is a secret-shaped pattern matcher and cannot reliably scan an arbitrary file's contents for meaning. For `Write`, `Edit`, `Read`, `Glob`, and `Grep` it keeps only an explicit allowlist of structural fields (path, byte counts, line counts); for `Bash` and `Agent` it keeps the actual text. The allowlist is the choice that matters: it is the opposite of a denylist, so a new field Claude Code adds later cannot silently leak. Anything else falls back to compact JSON with long content-shaped values omitted.

`recorders.py`. Where everything meets: `build_row` is pure (no clock, no I/O, parameters only) and `record` is the impure wrapper that parses, loads policy, builds, and appends. Every free-text field goes through the same order, and the order matters: never-send check first (if it fires, every excerpt on the row becomes the literal string `[never - send]` and nothing else runs), then `normalize`, then `redact`, then `truncate`, never the other order, since truncating first could cut a secret in half at the boundary and leave the remaining half unrecognizable to any rule. It hashes the working directory with FNV-1a (a fast, non-cryptographic hash) rather than SHA-256 (a cryptographic hash), a plain integer loop that needs no import, since the field only needs to group rows, not resist attack. If the policy file is broken, `_load_policy_fail_open` substitutes the packaged default rather than losing the row.

`sslctx.py`. Builds a TLS (Transport Layer Security, the encryption HTTPS runs on) context that verifies certificates and repairs the one case where the development machine's python3 has an empty certificate store (D-012), by loading a fallback bundle from one of three common operating-system paths. It is not used anywhere on the recording path; nothing in M1 opens a socket except `verdict doctor`'s optional probe. It never disables verification; if no fallback bundle exists, every connection through it simply keeps failing closed.

`span.py` (new in M2). Turns one session's ledger rows into a Span: the evidence the Stop verifier will reason about, covering the current prompt plus earlier prompts walked back to the first clean stop, 5 prompts, or a `clear` (D-020). It matches a failure to a later success by tool and by a normalized command or path, so a fixed test is not counted as still failing, and it tracks which failures a past verdict already acknowledged above a threshold (`t_ack_hi`) so they are not re-flagged. Every field is read defensively with `.get()` and a type check; the module is Hypothesis-tested to never raise on an arbitrary or adversarial row list, so a corrupt ledger degrades to an empty span rather than crashing the verifier that has not been built yet.

src/agent_verdict/, the command line tool

`__init__.py`, `cli.py`. The installed `verdict` command. `cli.py` parses the command line and dispatches to `stats` or `doctor`, reusing each subcommand's own flag parser as an `argparse` parent so a flag is only ever declared once. A bad subcommand name just prints `argparse`'s own usage error and exits non-zero; nothing here touches the ledger directly.

`stats.py`. Summarizes the local ledger: sessions, prompts, tool rows, failures, stops, how many stops carried a claim, never-send rows, and redaction hits. It is split into a pure aggregator (`compute_stats`, testable with injected rows) and a thin loader that reads through the same `ledger.py` code the plugin wrote with. An empty or missing ledger just reports all zeros; it does not error.

`doctor.py`, `_doctor_checks.py`, `_doctor_interpreter.py`. One command split across three files to stay under the line-count guideline. It reports which interpreter `run.sh` would actually pick, whether `~/.``verdict`'s permissions are correct, whether the plugin is registered, recent `hook.log` outcomes, and (optionally, `--no-probe` to skip it) whether each provider host is reachable over an encrypted (TLS) connection. Only a missing interpreter, a permission violation, or (with `--audit`) a recorded exception affects the exit code; an unreachable provider or an absent API (application programming interface) key is informational only, because the collector must be diagnosable on a machine that has never talked to a provider. `--fix-interpreter` probes candidates inside their own subprocess, since the certificate-store problem is per-interpreter, and writes the winner to a private file `run.sh` reads.

scripts/, development-time tooling, none of it ships

`sync_hot.py`. Copies `plugin/hooks/verdict_hot/` verbatim into `src/agent_verdict/verdict_hot/`, the only sanctioned way the two copies ever change together. `make check` fails the build if the committed trees ever differ, so a forgotten sync is caught before merge, not at install time.

`gen_redact.py`. Generates `_redact_rules.py` from the vendored `gitLeaks.toml`, translating Go's RE2 syntax to Python's (scoped inline flags, `\z` to `\Z`, delimiters turned into lookaheads so they are asserted but never consumed). Every translated pattern is compile-checked under real Python 3.9 semantics; the generator refuses to finish if fewer than 150 of about 222 vendored rules survive translation, rather than silently shipping a shrunk rule set.

`gen_corpus.py`, `gen_corpus_negatives.py`. Generate the redaction test corpus: 288 synthetic positive secrets across 18 families and 12 contexts (two thirds unlabeled), and 492 negatives across 40 categories that must never be redacted, each pre-classified as "evidence text" or "opaque token" from its category alone, never from the redactor's actual output, which is what makes the D-027 false-positive split measurable rather than circular. Both are seeded, so a re-run reproduces the same corpus.

`bench_hook.py`. Measures real end-to-end hook latency by spawning the actual `run.sh` launcher per fixture (D-029's gate: p50 at most 75ms, p95 at most 150ms). It reports the bare-interpreter floor beside every number, and its methodology comment records a real measurement trap: sending a child's output to `DEVNULL` added a flat 18 to 20ms from a Python subprocess polling fallback, fixed by capturing output instead. A failing gate fails the `make` target, not the plugin.

`e2e_cheap.py`. Runs one real, billed, headless Claude Code session to prove the plugin actually works once installed (asks it to run a command that exits 3, then checks the resulting ledger for the expected rows). The docstring warns it

should be run at most a few times during development because each run is a real API call; it always uses a temporary working directory and a temporary VERDICT_HOME, never your real data.

smoke_jev.py. Benchmarks the M2 model providers with a realistically shaped payload, entirely with the standard library, before any product code depends on one. Its instructions text always ends by telling the model that the untrusted section is data, not commands (the prompt-injection defense behind D-010). gate_passes requires the returned model id to exactly equal the requested one; a looser "contains" check was replaced (D-024) the first time a real response showed why it was unsafe. If no provider key is set in the environment, it skips that provider rather than failing.

check_name.sh, capture_tasks.sh, fixture_capture_plugin/, process_fixtures.py. A small toolchain for keeping the test fixtures honest: check_name.sh checked the project name was free on PyPI, npm, and GitHub; capture_tasks.sh drives four throwaway headless sessions loaded with fixture_capture_plugin, a six-line plugin whose only job is to dump each hook's raw stdin to a file (its own manifest says "Never ship"); and process_fixtures.py sanitizes those raw captures (replacing the real home directory, truncating long strings) into the committed, named fixtures under tests/fixtures/hooks/, with a provenance file recording exactly when and with which Claude Code version they were captured. If a capture run is interrupted, the raw payloads are simply incomplete and get re-captured; nothing partial is ever committed directly.

4. One real event, end to end

The fixture tests/fixtures/hooks/post_tool_use_failure_bash.json captures a real session where Claude ran a shell command that prints to standard error and exits 3.

The incoming payload (abridged):

```
{
  "hook_event_name": "PostToolUseFailure",
  "session_id": "0e72a50a-26a8-435d-8708-523dd0eecddd",
  "prompt_id": "db5b98cd-88f7-422a-9c10-bc3d487e5d47",
  "tool_name": "Bash",
  "tool_use_id": "toolu_0123j6wgsDxTvNRbm4Eo8NEe",
  "tool_input": {"command": "sh -c 'echo boom >&2; exit 3'"},
  "error": "Exit code 3\nboom",
  "duration_ms": 410,
  "is_interrupt": false,
  "permission_mode": "acceptEdits"
}
```

The ledger row it becomes (from the long walkthrough's section 7, produced by calling recorders.build_row directly against this fixture):

Field	Value	Meaning
schema_v	1	Ledger format version.
event	post_fail	A tool call failed.
session_id	0e72a50a-...	Also the ledger file's name.
prompt_id	db5b98cd-...	Which user prompt this belongs to.
tool_name	Bash	Which tool failed.
input_excerpt	sh -c 'echo boom >&2; exit 3'	The command, redacted and capped at 300 chars.
status	error	Fixed by the event firing, D-002, never a guess.
exit_code	3	Parsed from the exact first line Exit code 3.
is_interrupt	false	It failed on its own; the user did not stop it.
error_excerpt	Exit code 3\nboom	The error text, redacted, head 300 / tail 2000.
duration_ms	410.0	How long the command ran before failing.

Field	Value	Meaning
is_check	false	Not a test, build, lint, or type-check command.
never_send	false	No credential path or command was touched.
redaction_hits	0	No secret-shaped spans were found.
redaction_failed	false	The redactor ran normally.

This row supports exactly one sentence: at this moment, in this session, under this prompt, a Bash command exited 3. If a later stop row in the same file claims the tests pass, the two become comparable. Making that comparison automatic is M2.

5. The safety rules, in one table

Rule	What it means	Where
Never store file contents	A file's actual content is never written to the ledger; only structural facts (path, size, line count) are kept for Read/Write/Edit.	<code>_tool_output.py</code>
Never-send list	If a tool call touches a credential-shaped path or command, every text field on that row becomes [never - send]; nothing is even looked at.	<code>gates.py</code> , <code>policy.py</code>
Redact before truncate	Text is redacted, then truncated, never the reverse, so a secret cannot be cut in half and become unrecognizable.	<code>recorders.py</code>
Fail open on the model path	If the redactor, the parser, or the recorder raises, the hook still exits 0 and the session is untouched; a missing row is preferred over a broken tool.	<code>verdict_hook.py</code>
Never print to stdout unless deciding	A hook's stdout can be read as a permission decision by Claude Code, so nothing is ever printed except a deliberate decision (none exist yet in M1).	<code>run.sh</code> , <code>verdict_hook.py</code>
Private 0700/0600 files	<code>~/.</code> verdict is mode 0700, every file in it mode 0600, re-asserted by explicit <code>chmod</code> regardless of <code>umask</code> .	<code>paths.py</code> , <code>ledger.py</code>
No network on the recording path	Nothing under the hot path opens a socket; verified by parsing a real import trace.	<code>sslctx.py</code> , tests
Kill switch	Setting <code>VERDICT_DISABLE</code> to anything non-empty disables recording immediately, checked before any other work, in both the shell script and the Python entry point.	<code>run.sh</code> , <code>verdict_hook.py</code>

6. Numbers that matter

Number	Value	Source
Tests passing at tag m1	672	<code>docs/DECISIONS.md</code> , D-030
Redaction recall, held-out probes	0.96 to 0.97 on the last three fresh probes	<code>docs/measurements/redaction-heldout-2026-09-21.md</code>
Redaction evidence-text false positives	0.011 (probe 4, the passing round)	<code>docs/measurements/redaction-heldout-2026-09-21.md</code>
Hook latency gate	p50 at most 75 ms, p95 at most 150 ms per event	<code>docs/DECISIONS.md</code> , D-029
Hook latency measured (post-fail, tag m1)	p50 58 ms on Python 3.14, 70 ms on Apple's Python 3.9	<code>docs/DECISIONS.md</code> , D-029/D-030

7. Glossary

Hook. A program Claude Code runs at a fixed moment in a session.

Event. One of the moments a hook fires on; Verdict records six of them.

Ledger. The append-only file of recorded events, one per session, under `~/ .verdict/events/`.

Row. One line of the ledger: one JSON object, one event.

Redaction. Replacing a span of text that looks like a credential with a marker before it is written anywhere.

Never-send. A stricter rule than redaction: when a credential file or command is touched, nothing on that row is kept at all, not even a redacted version.

Exit code. The number a command returns; zero conventionally means success. Verdict treats it as a fact, never a model's guess.

Shadow mode. The default mode: record everything, judge nothing, block nothing.

p50. The median of a set of measurements: half the runs were faster than this.

Held-out probe. A test set built by someone other than the implementer, with material never seen before, so a fix cannot be tuned to the very test that measures it.

Condensed from docs/walkthrough/code-walkthrough.md. Every fact here was checked against the source in that longer document or against the code directly.